

09h — Database Migrations

Key Concepts

Term	Definition
Migration	Versioned, code-managed change to database schema
Up Migration	Applies the change (add table, add column)
Down Migration	Reverses the change — used for rollback
Schema Drift	Different environments have different DB structures — caused by out-of-sync migrations
Migration History Table	DB table that tracks which migrations have already run (e.g., <code>__EFMigrationsHistory</code>)

Why Migrations Over Manual SQL

Manual SQL	Migrations
No history of changes	Full versioned history
Hard to sync across environments	Same migrations run on dev/staging/prod
Team members get out of sync	Everyone runs migrations, stays in sync
Rollback is manual and risky	Down migration reverses changes cleanly

How It Works

```
Migration 001 – Create Users Table      (Up / Down)
Migration 002 – Add email column        (Up / Down)
Migration 003 – Add index on email      (Up / Down)

DB tracks which have run in __EFMigrationsHistory
Roll back Migration 003 → runs its Down method
```

EF Core Commands (.NET)

```
dotnet ef migrations add AddEmailToUsers # create migration
dotnet ef database update                 # apply all pending
dotnet ef database update AddUsersTable  # roll back to specific point
```

Migration Tools by Language

Language	Tool
.NET / C#	Entity Framework Core
Node.js	Prisma (<code>prisma migrate dev</code>), Knex.js
Python	Alembic (SQLAlchemy), Django (<code>manage.py migrate</code>)
Java	Flyway, Liquibase
Ruby	ActiveRecord (Rails)

Best Practices

- Never edit a migration after it has been applied to any environment
- Always test the Down migration — rollback must work in production
- Keep migrations small and focused — one change per migration
- Run migrations as part of CI/CD pipeline — schema and code deploy together
- Never delete old migration files — they are the schema history

Safe Large Table Migration Pattern

1. Add column as **nullable** first (no lock, no default required)
2. Deploy application code
3. Backfill data in background job
4. Add constraints/indexes after backfill completes

Industry Examples

- **GitHub** — adds nullable columns first then backfills to avoid locking massive tables
- **Shopify** — no blocking migrations during peak traffic; background jobs for backfills
- **Any .NET shop** — EF Core migrations run automatically via `database update` in CI/CD

Interview Questions

Question	One-Line Answer
What is a database migration?	Versioned, code-managed schema change — like Git for your database structure
Up vs Down migration?	Up applies the change; Down reverses it for rollback
How does EF Core track migrations?	<code>__EFMigrationsHistory</code> table records which migrations have run
What if you edit an applied migration?	Other environments won't re-run it — causes schema drift and inconsistency

Question**One-Line Answer**

How do you safely add a column to a large prod table?

Add as nullable first, deploy, backfill in background, then add constraints